

EB 级大规模存储的数据面设计与思考

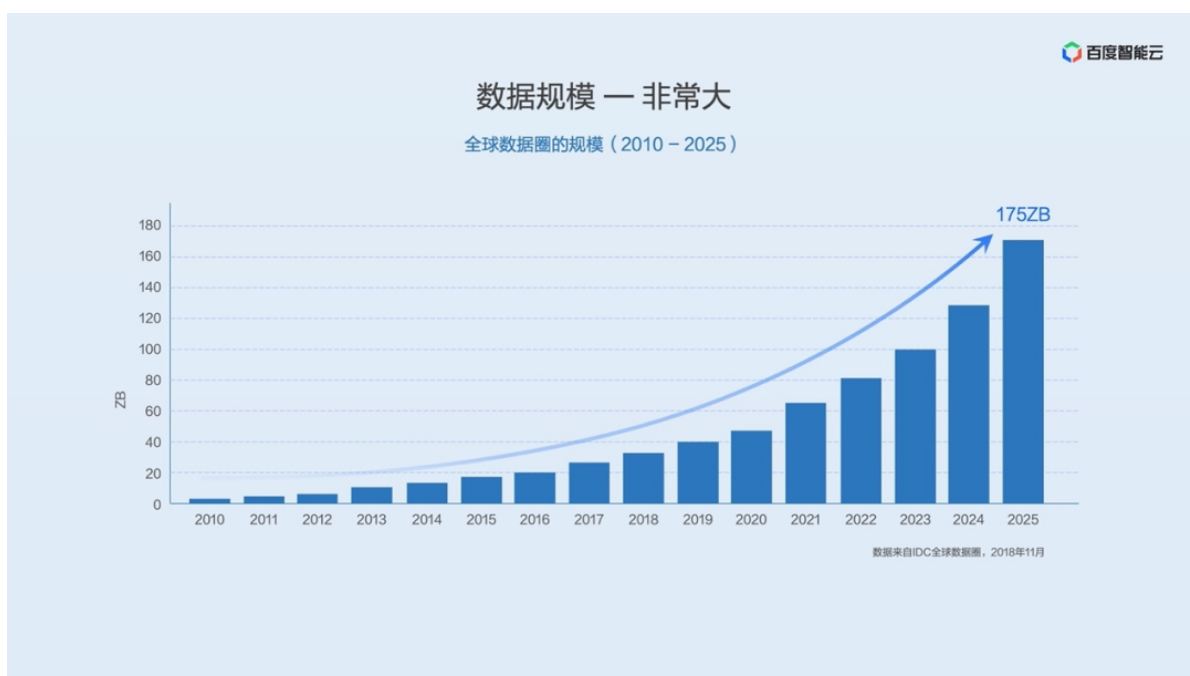
本文根据《面向百度网盘的大规模数据面存储架构思考与设计》线上分享整理而来。这是百度沧海·存储的数据面存储底座 ARIES 的第一次公开分享，完整地涵盖了从存储系统的设计思想、ARIES 架构设计、关键概念和工程实现挑战等环节。所以全文有点长，建议先收藏再慢慢阅读。关于百度沧海·存储的元数据存储底座 TafDB 的详细介绍，文章文末“延伸阅读”。注释：ARIES, A Reliable and Integrated Exabytes Storage。

“全文总共分为三个部分，第一部分对大规模数据存储进行概述；第二部分以百度网盘为例介绍百度沧海如何支撑 EB 级大规模存储；第三部分分享一些相关的经验与思考。”

大规模数据存储概述

1.1 数据的规模

IDG 曾经对全球数据增增情况做过一次预测，全球数据规模不仅越来越大，而且保持加速增长的趋势，如下图所示：



在这张图中，数据规模的单位都是 ZB，而 1ZB 等于 1024EB，1EB 等于 1024PB，从这个单位的量级可以看出，当今数据的规模是相当庞大的。即使是从单体业务的角度去看，我们几年前还在使用 PB 作为规模的衡量单位，但近几年，衡量单位已经上升到了 EB，所以本文将会以百度网盘这个存储服务作为例子，介绍百度沧海·存储如何支撑网盘的 EB 级存储，同时分享一些经验与思考。

1.2 数据的分类

数据的模型不一，种类繁多，应用场景千差万别，我们从一个工程人员的视角出发，将数据进行分类，如下图所示：

数据分类 — 一个工程人员的视角



在上述分类框架中，总共分为三层，自上而下分别是：

- 应用层
- 结构层
- 基础层

最底层是基础层，基础层的数据类型非常简单，分别抽象为**数据项**、**数据段**以及**数据流**，几乎所有的数据类型，往最底层拆解，都可以认为是由这几种类型构成。

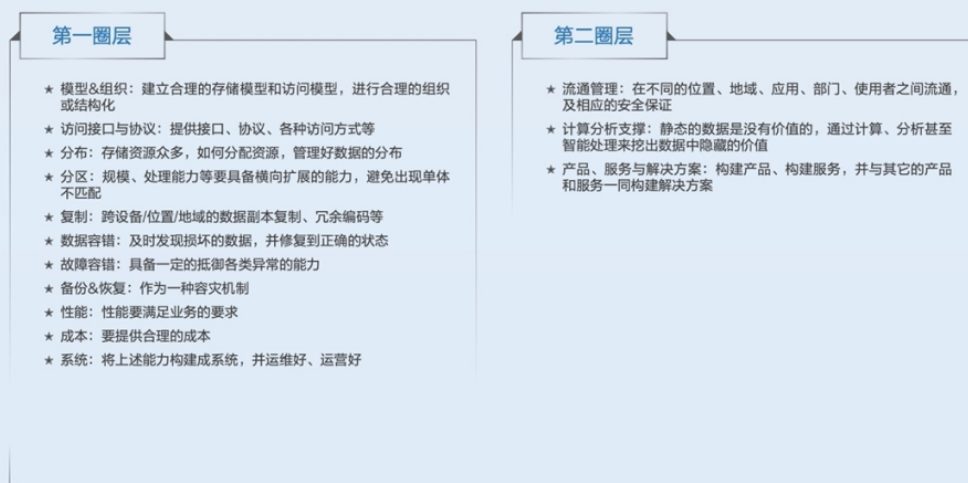
再往上是结构层，在这一层，基于工程需求，通过各种或强或弱的结构性，来表达基础数据类型之间的关系，构成更加复杂的数据类型，包括非结构化、结构化甚至半结构化。非结构化类型包括文件、对象、块、二进制流和文本流等；结构化和半结构化增更加丰富，包括了数据结构、关系数据、键值数据、表格数据、文档数据、时序和时空数据、图数据等等，此类数据有时候也是通过非结构化类型来表达的。

最上面一层是应用层，在这一层，数据的类型就更加丰富了，且往往与应用场景强相关。这一层常见的数据类型包括媒体、图片、纯文本 / 日志、word / PDF、可执行文件、数据库文件、压缩文件等等。

1.3 大规模数据存储的范畴

我们谈论大规模数据存储的时候，我们在谈论什么？下图是我们的回答：

大规模数据存储的范畴



我们认为，大规模数据存储的范畴可以分为两个圈层，第一个圈层是存储领域比较核心的技术要素，第二个圈层是基于存储而产生的相应的各种数据管理与服务要素。

在第一圈层中，首先要对各类数据建立合理的**存储模型**和**访问模型**，并进行相应的组织和结构化，同时也要提供访问接口与协议，提供各种访问方式。存储系统需要管理数据的分布、分区以及解决数据的复制或冗余需求，包括基于多副本的传统复制形式和基于冗余编码技术的冗余形式。

数据存储在介质中，有一定的概率会出现损坏或丢失，需要能够及时发现损坏和丢失的数据，并将其修复和恢复到正确的状态，也需要对异常的数据和异常的环境进行容错，为了能够容灾，往往还需要具备**备份与恢复能力**。

成本和性能是大部分工程领域都需要面对的重要问题，更不用说大规模存储了。在很多领域，成本和性能呈现出一定程度上的此消彼长的跷跷板关系，存储领域也往往如此。如何在成本和性能之间进行折衷，以合理的成本提供合理的性能，尽量以**较低的成本**满足业务的性能需求，是一种非常关键的能力。

最后是系统化，当具备了上述的各种能力之后，如何将这些能力构建良好的系统，并持续稳定地维护和运营，方便用户和客户的使用，也是存储领域非常有挑战性的一个问题。

第二圈层主要包括流通管理、计算分析的支撑以及产品服务与解决方案。

数据往往会在不同的位置、地域、应用、部门、甚至使用者之间进行流通，这个流通过程需要得到良好的管理和安全保证。从数据的消费角度来看，除了备份需求，静态的数据是没有太大的价值的，如何消费数据以支撑高效的计算与分析，甚至智能化的处理，据此来挖掘数据中隐藏的价值，是存储生态中非常关键的一环。从数据的产品化和服务角度来看，如何将存储系统构建成为完善的产品，提供可信赖的服务，甚至与其它类型的产品和服务进行协同，构建出场景化的解决方案，是存储领域走向商业化的必由之路。

1.4 大规模数据存储面临的挑战

在任何工程领域，都会存在相应的技术挑战，大规模存储领域也不例外。除了相应的“硬核”技术挑战，本文也介绍了组织与文化方面带来的“软性”挑战，如下图所示：

大规模数据存储的挑战



第一个方面是来自于客观世界的技术挑战，抽象为四个方向，分别是

- 行为
- 环境
- 资源
- 折衷

行为代表的是数据的访问特征与模式。在不同的场景中，数据的访问特征与模式也是各不相同，甚至是千差万别。不同的特征与模式，会给存储系统带来不同的需求与技术，也会带来不一样的挑战。

环境代表的是环境（尤其是分布式环境）的复杂性及其问题。不管是机器世界，还是机器运行所在的现实世界，其环境都相当复杂，甚至不可信赖，会导致很多的问题，而这些问题有进一步给存储系统带来相当大的挑战。

资源是基础，不仅资源自身存在一定的特性局限，资源的供给与管理也是一个不容忽视的问题，也会给存储系统带来不小的挑战。

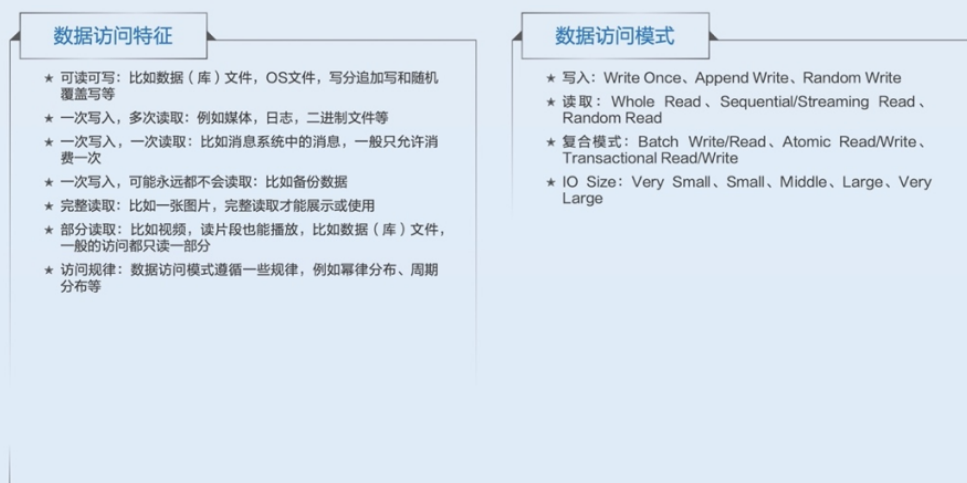
最后是折衷，所谓折衷，指的是如何在多个问题上进行权衡取舍。上文提到过，在很多工程领域，都存在成本和性能的折衷，在存储领域，参与折衷的问题点则会更多，至少，数据的可靠性会参与到这个折衷里面来。更多的折衷点也会大大提升其难度，带来更多的挑战。

第二个方面是组织与文化带来的挑战，包括团队的经验与能力、组织的需求及优先级、组织架构的影响和组织的文化风格等。

1.4.1 行为挑战

行为包括数据访问特征和数据访问模式，如下图所示：

客观世界的挑战 — 行为



不同的应用场景，抑或不同的数据类型，其数据的访问特征也有所不同，常见特征如下：

- 可读可写：数据可以被修改，也可以随时被读取，比如数据（库）文件、操作系统内的一些文件等
- 一次写入、多次读取：大部分数据都具备该特征，比如媒体和日志之类的数据，一般一次性写入，不再修改（或者通过重新写一份的方式来实现）修改，但可以多次被随时读取
- 一次写入、一次读取：典型如消息系统中的消息数据，一般而言，只会被消费一次
- 一次写入、可能永远都不会被读取：典型如备份数据，若没有产生恢复需求，备份数据在生命周期中往往一次都不会被读取，笔者也希望如此
- 完整读取：比如一张图片，或者一张发票文件，往往需要被完整地读取出来才能被使用
- **部分读取**：一些类型的数据可能只需要部分读取就可以应用起来，比如视频流和音频流，从任意一帧开始都可以播放，又比如数据库文件，业务的绝大部分访问都只是访问其中的一部分数据
- 访问规律：一些数据或数据系统在被访问时，存在明显的规律性特征，比如**幂律分布**、**周期分布**等

同样地，在不同的方面，数据也存在很多不同的访问模式，包括：

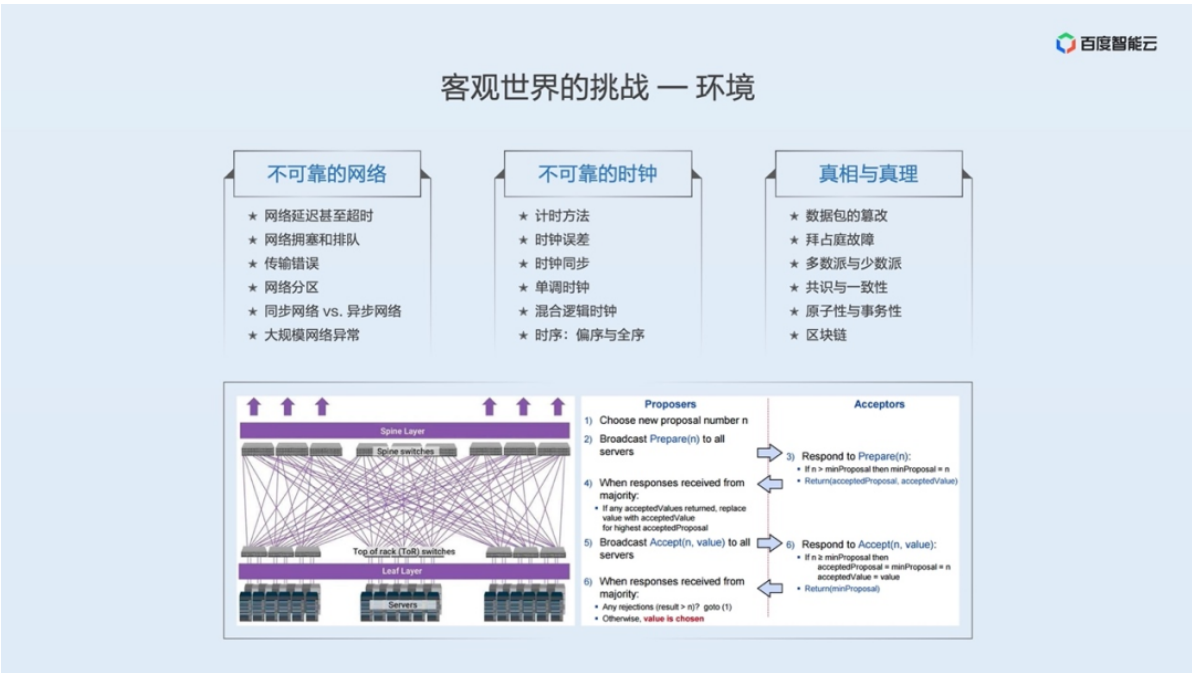
- 写入方面：其模式包括一次性写入、追加写入和随机（覆盖）写入等
- 读取方面：其模式包括一次性完整读取、顺序或流式读取以及随机读取等
- 复合模式：包括批量读 / 写、原子读 / 写，事务性读 / 写等
- I/O Size 方面：从极小粒度 I/O，到小粒度 I/O、中等粒度 I/O、大粒度 I/O 甚至极大力度 I/O，不一而足，在不同的场景中，粒度大小的边界也不一样

面对不同的访问特征和模式，存储系统可能需要有针对性地，提出不同的设计，实现不同的方案，实施不同的优化手段，以应对相应的挑战。

1.4.2 环境挑战

环境挑战指的是环境的复杂性及其问题带来的挑战。绝大部分大规模存储都是分布式的，面临的是分布式环境，而分布式环境比单机环境要复杂得多，导致的问题也多很多，这也是大规模存储面临的最大的挑战。

参考 Martin Kleppmann 博士《Designing Data-Intensive Applications》一书，这些挑战可以被高度总结为三个方面，具体如下图所示：



第一个方面是**不可靠的网络**。分布式环境中，节点之间需要网络进行通信，而网络自身也存在诸多不可靠的因素，比如网络可能会出现偶发的延时和超时，也可能出现一定程度的拥塞和排队，甚至可能出现网络分区；同步网络和异步网络带来的挑战也所有不一样；如果网络出现大规模异常，又该如何面对等等，可以看出，不可靠的网络带来的挑战可谓相当之大。

第二个方面是不可靠的时钟。时钟一个重要作用是计时，首先要设计计时的方法，需要分析计时方法的误差大小，如何在多个节点之间同步时钟以降低误差，如何基于时钟及其误差实现时序关系等，并不容易。对于一个物理时钟，至少要提供单调递增的时钟输出作为基础，以简化应用程序的设计。事件之间，什么时候需要全序关系，什么时候仅需要偏序关系，需要仔细地分析，不同时序关系，对时钟的需求也不一样。单机的物理时钟，即使能够保证单调性，能够实现单机层面的全序关系，但难以实现分布式层面的全序关系，这满足不了一些分布式系统的需求，例如在分布式数据库场景中，一些方案依赖全局层面的单调时钟，实现全系统事件之间的全序关系，因此而提出了逻辑时钟和**混合逻辑时钟(HLC)**的机制。可以看出，在分布式领域，时钟是一个非常棘手的问题，对分布式系统带来的挑战也非常大。

第三个方面是**真相与真理**。网络上一个数据包在发送的时候被第三方截获并篡改了，那么目的地收到的消息可能是一个错的或者假的，会引发安全问题，甚至可能会引发拜占庭故障这样的复杂异常。一个分布式环境有很多节点，那么存在多数派和少数派的认知问题，如何在节点之间形成共识，达成一致，并不容易。如何克服网络环境的不可靠实现跨节点的原子性和事务性处理，也是一个不小的挑战。近年来，以区块链技术为代表的新型去中心化跨节点解决方案也在得到更多的应用。

笔者在上面的图中提供了两个小的示意图，左下角的小图是数据中心里面常见的一种网络架构图，右下角的小图是 Paxos 算法的过程描述。网络架构图代表分布式环境极其复杂性，算法描述图代表研究人员和软件开发者如何去应对这种复杂分布式环境带来的挑战。

1.4.3 资源挑战

资源是大规模存储的基础，除了存储，资源还包括计算、网络和能源等多种类型，但在本文中，资源特指存储资源。大规模的数据最终要落地到多种多样的存储资源中实现持久化，存储资源的挑战来自于两个方面，一个是资源的管理，另一个是**存储介质自身的局限性**，如下图所示：

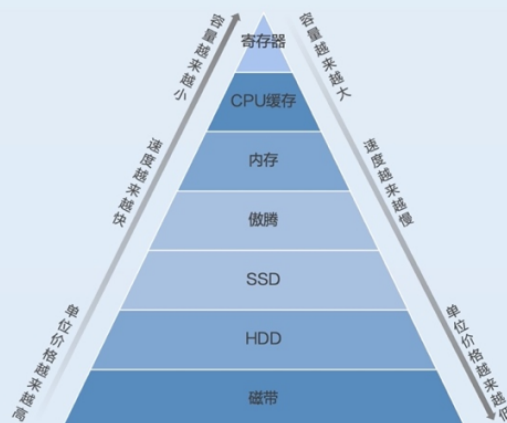
客观世界的挑战 — 资源

资源管理的挑战

- ★ 机器、机房、数据中心基础设施保障
- ★ 资源运营挑战，资源调度挑战等

存储介质的局限

- ★ 存储介质的性能与成本成反比
- ★ 以磁盘为例，目前产业界更优先去解决的还是容量和 TCO 问题，性能问题目前解决得不是很理想
- ★ SSD 想要取代更多 HDD 场景，性价比还有很长的路要走
- ★ 傲腾介质有诸多优势，但成本问题仍然存在解决空间
- ★ 磁带成本低廉但有访问模式的限制，不过依然活跃
- ★ 总结：各种类型的介质都存在其优势和局限



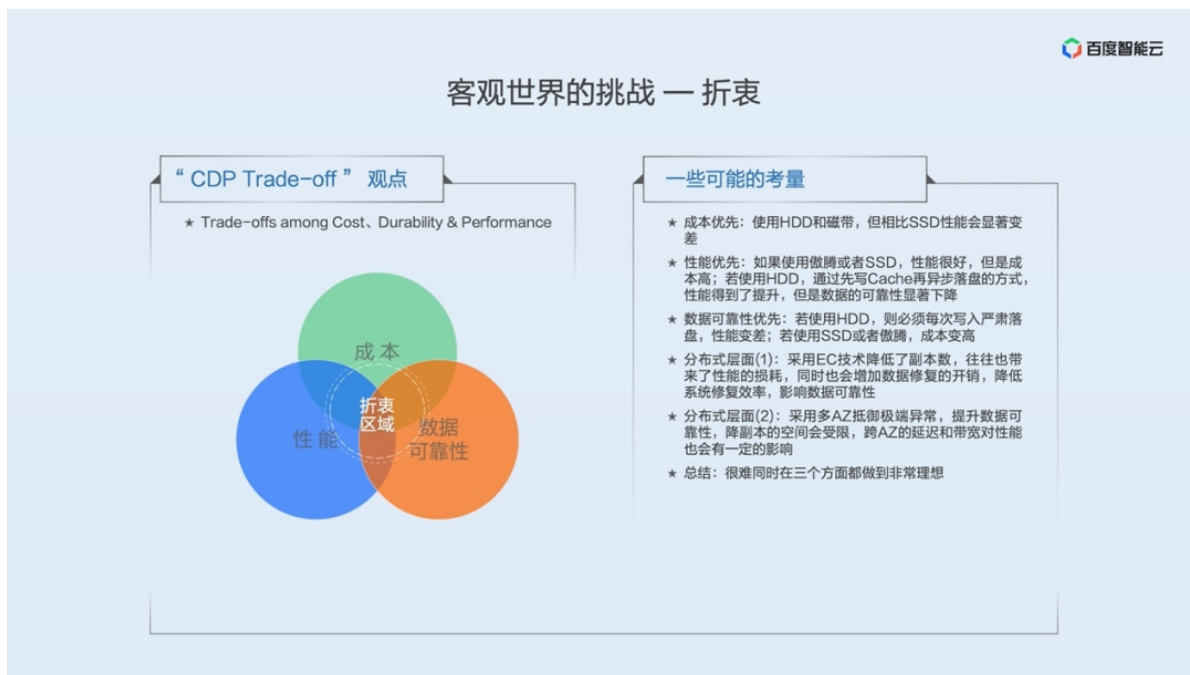
1 ns	L1 Cache Ref	← 1 sec
4 ns	L2 Cache Ref	← 4 sec
100 ns	DRAM	← 100 sec
16,000 ns	SSD	← 4.4 hours
2,000,000 ns	HDD	← 3.3 weeks
~50,000,000 ns	Network Storage	← 1.5 years
1,000,000,000 ns	Tape Archives	← 31.7 years

第一个方面是资源管理的挑战。一个存储系统，并不是简单地把数据存到存储介质这么简单，存储系统之下还会依赖很多基础设施层面的东西，比如介质、机器与机房的供应和管理，数据中心基础设施的能源和网络保障等。只有这些资源和基础设施得到了良好的管理和保障，整个存储系统才有可能长期稳定地运行。此外，资源运营和调度也是非常关键的一环。对存储系统而言，这些资源管理方面的工作是非常基础但又必须面对的挑战。

第二个方面是存储介质自身的局限性带来的挑战。存储介质除了自身的典型特征之外，一般多少都会存在一些局限性。这些特征和局限性里面，很重要的两个方面是成本和性能，二者之间呈现跷跷板的反向关系。从介质层次图中从上往下看，从寄存器和 CPU 缓存这种非常高性能的介质到下面的 SSD、磁盘、磁带，介质的访问速度变得越来越慢，但介质的单位价格也越来越便宜。从下往上看，介质访问速度越来越快，但其单位价格也会越来越贵。实际上，每种介质都存在特有的优势和局限，存储系统需要根据需求，充分发挥好这些介质的优势，尽量避开它们的局限。

1.4.4 折衷挑战：CDP Trade-off

笔者在这里提出“CDP Trade-off”的观点，这里 C 代表成本（Cost），D 代表数据可靠性（或持久性，Durability），P 代表性能（Performance），如下图所示：



AZ: Availability Zone

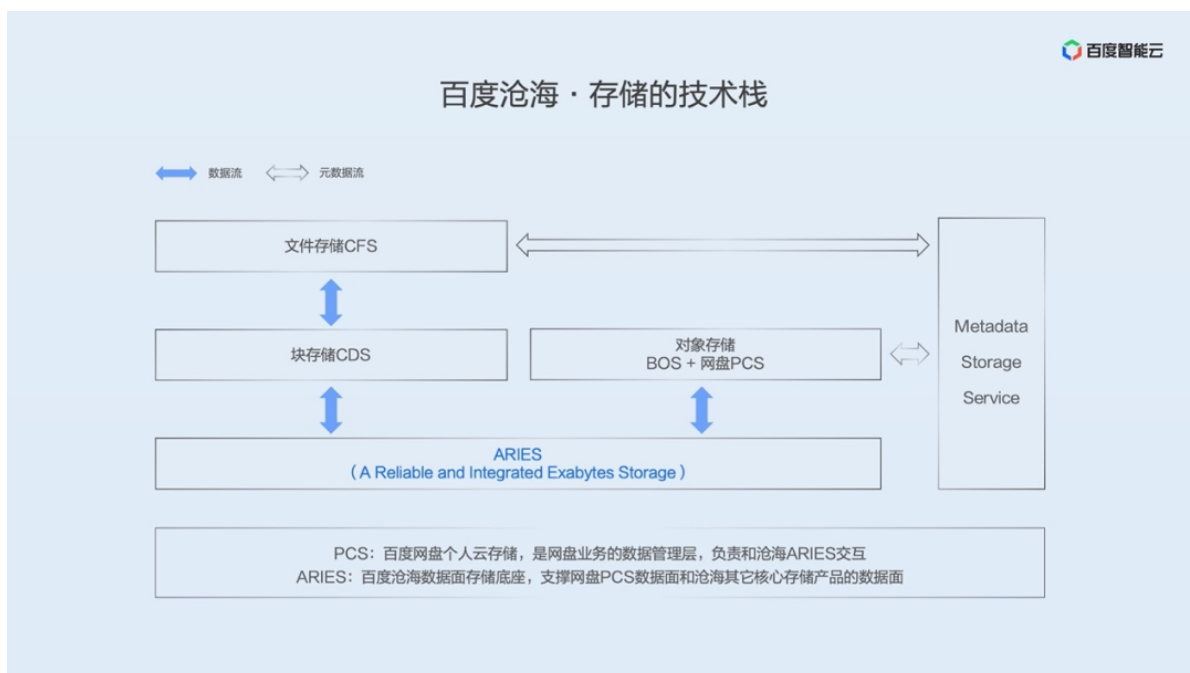
笔者认为，在很多时候，存储系统需要在 C、D 和 P 之间进行折衷，图上也给了一些折衷考量的点，这是我们在研发真实存储系统的时候很可能会遇到的一些挑战。总体上来看，存储系统还是难以同时在这三个方面都做到非常理想的程度，多少会存在一些取舍。

百度沧海的大规模存储架构——以网盘为例

2.1 百度沧海·存储体系简介

在这部分，本文以百度网盘为例，介绍百度沧海如何实现大规模存储并成功支撑百度网盘的存储需求。在此过程中，也会比较详细地介绍系统的概念的设计以及系统如何应对上一部分介绍的各种挑战。

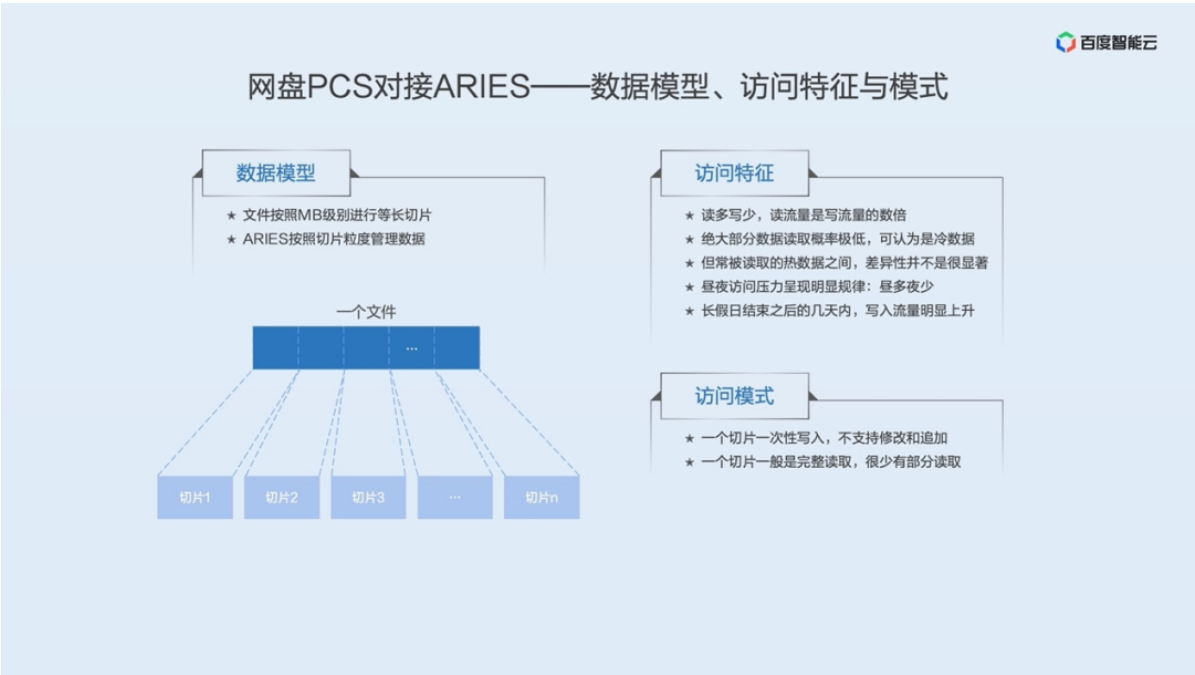
这里展示一张百度沧海存储体系的简要示意图：



从图中可以看出，最底层是一个名为 **Aries** 的系统（全称为 **A Reliable and Integrated Exabytes Storage**），它是**百度沧海的数据面存储底座**。它向上支撑网盘的 PCS 层（全称为 **Personal Cloud Storage**），以及沧海自身的对象存储 BOS 和块存储 CDS。块存储 CDS 又支撑了文件存储 CFS。当然 CFS 和 BOS 自身还有复杂的目录树和元数据，这个是由沧海的元数据底座服务提供的支撑。本文的重点是网盘 PCS 和沧海 Aries，网盘 PCS 是网盘业务中的核心存储层，通过与沧海 Aries 交互，完成了最终的数据存储和管理职责。

2.2 Aries 系统的概念与设计

首先以网盘为例，介绍一下业务如何对接 Aries，如下图所示：

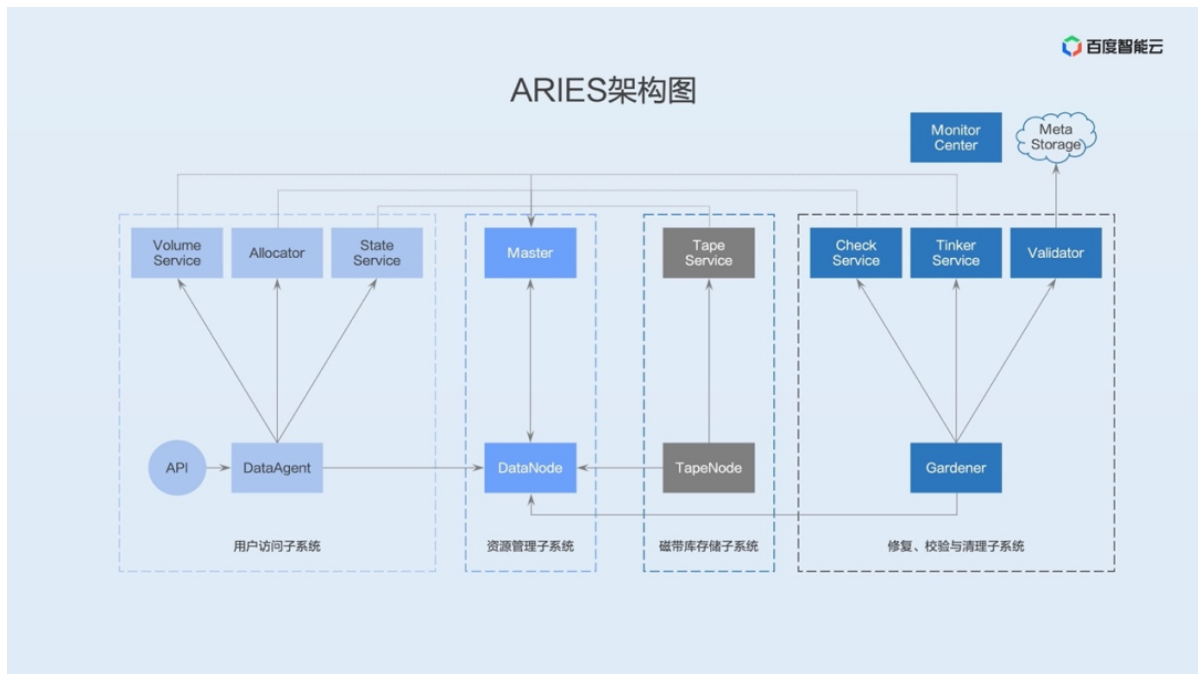


在数据模型层面，网盘 PCS 层将用户上传的文件按照 MB 级别的大小进行有序切片（**目前默认是 4MB**），然后将**每个切片作为 Aries 管理数据的基本单位**。

在数据访问特征层面（需要说明的是，这里的访问特征仅指代指网盘的数据落到 Aries 系统的访问特征，就不代表网盘业务自身在用户侧的访问特征），存在几个明显的特点：第一，**读多写少**，读流量是写流量的数倍大小；第二，**绝大部分数据的读取概率都比较低**，可以认为冷数据占主要部分；第三，即使是偶尔会被读取的非冷数据，它们之间的**访问热度差异**也并不是很显著；第四，**访问压力呈现明显的昼夜变化规律**，白天访问压力大，夜间访问压力小；第五，在长假日结束之后的几天内，写入流量会出现一个明显上升的过程，这个很容易理解，因为大家出去玩了之后拍了很多照片，回来之后很多人会备份照片到网盘。

在数据访问模式层面，存在如下几个特点：第一，一个切片，一次性写入，不支持追加也不支持修改；第二，一个切片一般是完整读取，很少有部分读取（虽然 **Aries 支持部分读取**）。

Aries 系统的架构如下图所示：



Aries 架构一共包含十几个模块，初看会有些复杂，但是采用了子系统化的设计思路，目前整个系统一共由四个子系统组成，不同的模块分属不同的子系统，下沉到不同的子系统之后再看则容易理解。四个子系统分别是：

- 资源管理子系统
- 用户访问子系统
- 修复、校验与清理子系统
- 磁带库子系统

资源管理子系统由 Master 和 DataNode 构成，主要负责存储资源管理以及集群的基础管理；用户访问子系统由左边的 DataAgent、VolumeService、Allocator 和 StateService 几个模块构成，主要负责**用户访问通路**；修复、校验与清理子系统由右边的 CheckService、TinkerService、Validator 和 Gardener 构成，顾名思义，其主要负责数据的完整性保证、可靠性保证和垃圾清理；磁带库子系统由 TapeService 和 TapeNode 这两个模块组成，专门负责对接磁带库存储。

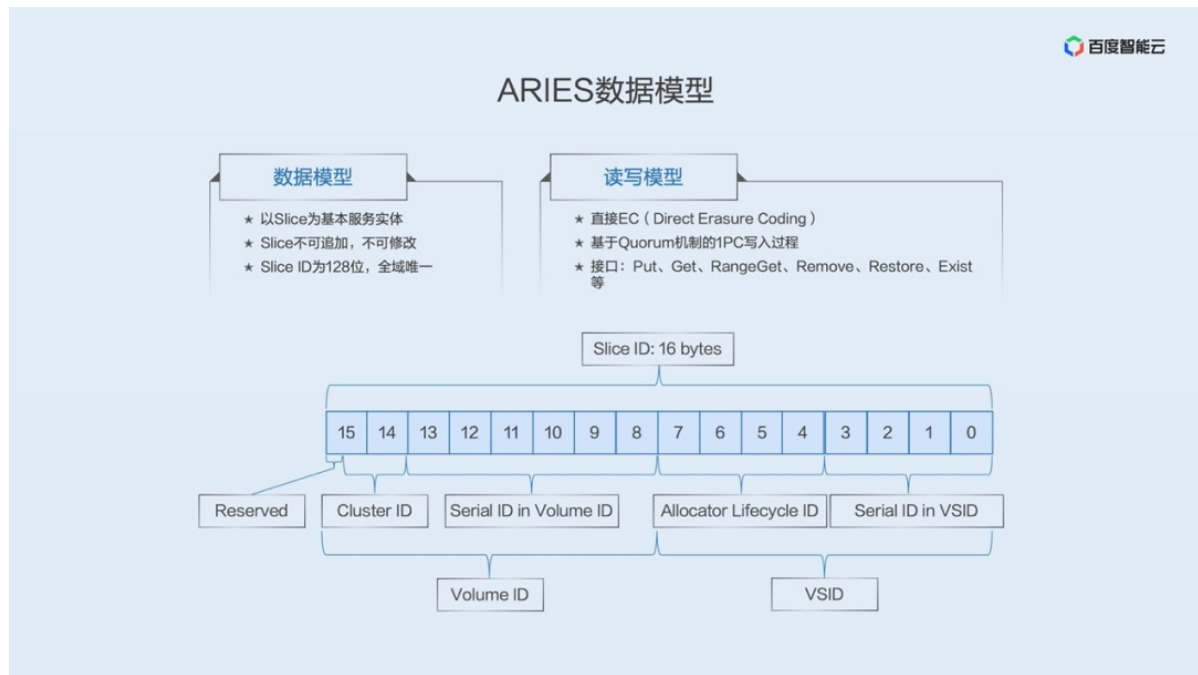


如上图所示，Aries 在架构设计上的关键思路包括如下几点：

1. Aries 基于微服务化和子系统化的设计，模块间高内聚低耦合架构，容易实现架构的扩展和演进；

2. 作为云存储的数据底座，Aries 承担存储领域更多的能力，实现更好的技术复用；
3. 面向故障设计，在 Aries 中系统的故障应对能力与系统的功能处于同等重要地位，二者都不可或缺；
4. 面向 EB 级别单体规模的设计，以 EC 模型为主，同时也支持多副本；
5. 全介质管理，Aries 能够管理从傲腾到 SSD、HDD，再到 SMR HDD，甚至磁带的多种类型介质，支持多引擎集成，针对不同的业务场景、不同的介质使用不同的引擎，系统能够运行在不同的状态下，充分体现 CDP Trade-off。

Aries 的基础数据模型如下图所示：

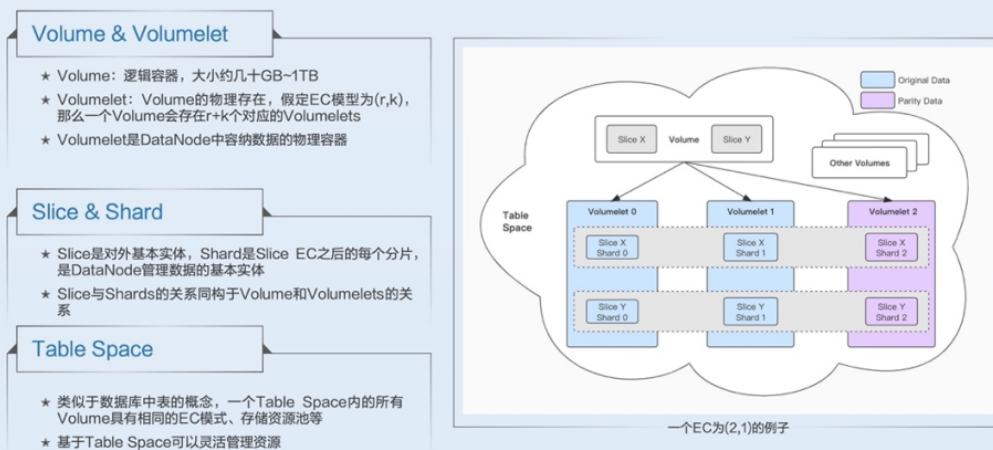


从数据模型上看，Aries 以 **Slice 为基本服务实体**，对应上述提到的文件切片，**Slice 不可追加，不可修改**。Slice ID 为 128 位，可以实现全域唯一。从图中可以看出，Slice ID 左边的高 64 位是 Volume ID，Volume 是一个容器概念，Volume ID 进一步划分为两个部分，高 16 位是集群 ID，低 48 位是集群内给 Volume 分配的的顺序 ID，所以 Volume ID 是全域唯一的。Slice ID 的低 64 位为一个 Volume 内部给 Slice 分配的 ID，这个 64 位进一步分为两个部分，其中高 32 位是分配进程的生命周期版本号，低 32 位是分配进程在这个版本下，给这个 Volume 的 Slice 分配的的顺序 ID。从读写模型上看，**Aries 采用的是直接 EC 模型** (Direct Erasure Coding)，即，一个 Slice 写进来的时候，直接对其做 EC 编码，然后将生成的分片写到各自对应的 DataNode 上。写入过程是基于 Quorum 机制的 1PC 方式。支持 Put、Get、RangeGet、Remove、Restore、Exist 等接口。

Slice就是4MB的文件切片吗？

Aries 的主要概念如下图所示：

ARIES主要概念



第一组概念是 Volume 和 Volumelet。Volume 是一个逻辑容器概念，容器大小一般在几十 GB 到 1TB 之间，Volumelet 是 Volume 的一个物理存在，假定 EC 模型为 (r,k) ，那么一个 Volume 会存在 $r+k$ 个对应的 Volumelet。对 DataNode 而言，Volumelet 实际上是容纳数据的物理容器，在不同的存储引擎里面，Volumelet 的形态并不一样。

第二组概念是 Slice 和 Shard。Slice 是 Aries 系统对外的基本实体，Shard 是 Slice 通过 EC 编码生成的各个分片，也是 DataNode 实际管理的基本数据单位。**Slice 和 Shard 的关系等同于 Volume 和 Volumelet 的关系。**

第三个概念是 Table Space，Table Space 是一个类似于数据库表的概念。一个 Table Space 包含多个 Volume，这些 Volume 具有相同的 EC 模式。Table Space 会绑定到一个 / 些资源池，基于 Table Space 的概念可以实现 Volume 和资源池的映射。

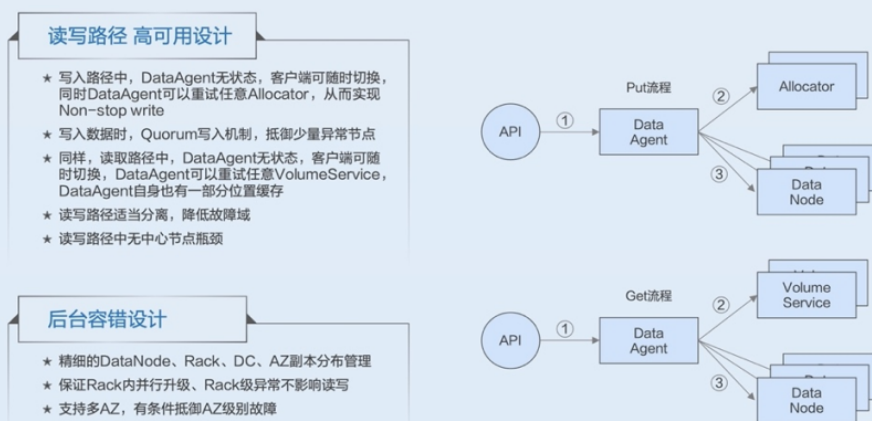
图中右侧以一个 EC 为 $2+1$ 的例子展示了这些概念之间的关系。图例中的 Volume 包含一个 Slice X，EC 之后生成了 Shard 0、Shard 1 和 Shard 2，其包含的另外一个 Slice Y 也是如此。Shard 0、Shard 1 和 Shard 2 分别存储在该 Volume 对应的 Volumelet 0、Volumelet 1 和 Volumelet 2 中。图例中的 Table Space 除了包含该 Volume，还包含了多个其它 Volumes。

2.3 Aries 如何应对挑战

2.3.1 高可用设计

高可用设计的相关机制如下图所示：

分布式环境挑战 — 高可用设计



第一个方面是读写路径的高可用设计，以图中展示的 Put 和 Get 的流程为例进行介绍。

Put 流程：Put 的时候数据首先从 API 发送到 DataAgent，然后 **DataAgent 访问 Allocator 服务获取 Slice ID**，最后再做 EC 编码并将各个 Shard 的数据分别写到对应的 DataNode 上面去。因为 Aries 的 Slice 是一次性处理的，**DataAgent 不需要管理会话状态**，是无状态的。那么在第一个步骤中，一旦 DataAgent 出现超时或者异常，API 可以立即重试其它的 DataAgent 继续 Put 流程。第二个步骤中，如果 Allocator 超时或者异常，DataAgent 可以立即重试其它的 Allocator 去获取一个新的 Slice ID，可以这么做的原因是，**Allocator 是多节点部署的，并且用了一致性 Hash 的方法来管理**。第三个步骤是写入 Shard 到 DataNode，基于 **Quorum 机制**，可以容忍少量异常的 DataNode 对写入的影响。可以看出，整个 Put 路径中，每个步骤都具备一些高可用的设计。

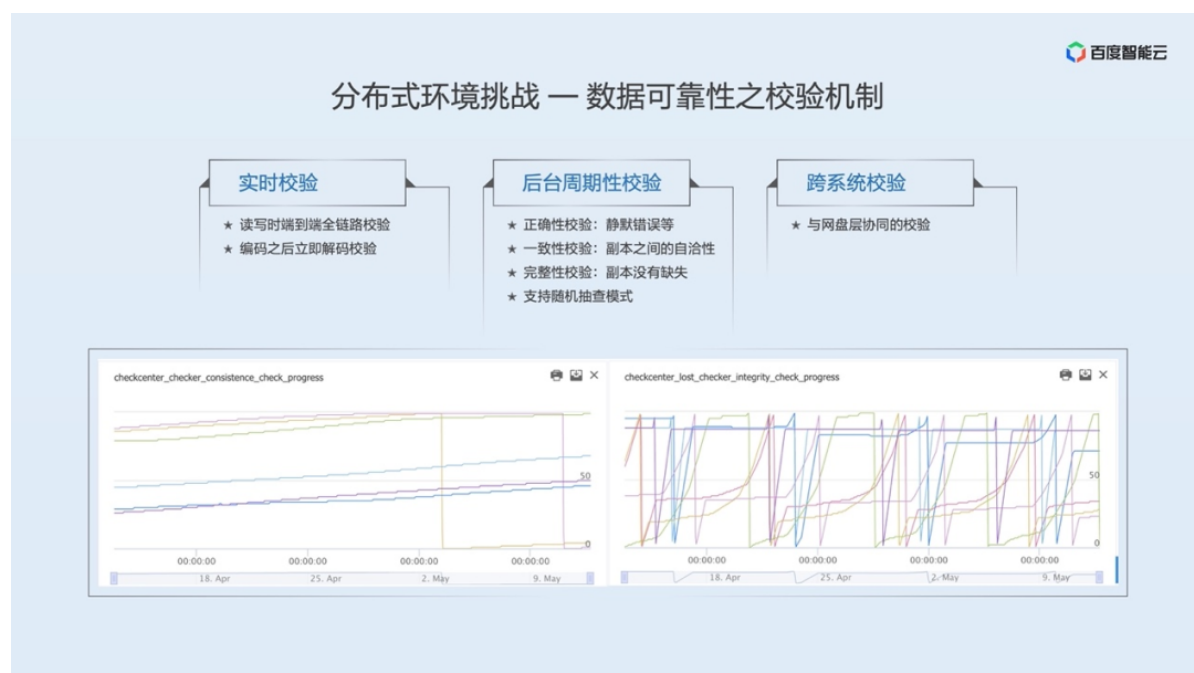
Get 流程，第一个步骤和 Put 流程是一样的，API 可以请求或者重试任意一个 DataAgent 以实现高可用，第二个步骤是 **DataAgent 从 VolumeService 获取目标 Slice 的（对应 Volume 的）分布信息**，多个 VolumeService 节点之间是**对等关系**，每个 VolumeService 都包含全部的分布信息，故 DataAgent 可以访问或者重试任意一个 VolumeService 以实现高可用。实际上，**DataAgent 也会尽量缓存一部分 Volume 的分布信息**，尽可能优化掉这个步骤，当然，缓存也有相应的失效和更新机制。第三个步骤是 **DataAgent 从相应的 DataNode 中读取足够的 Shard 并解码出原始数据**，因为 EC 的特性，一个 (r,k) 的 EC 模型，最少只需要 r 份数据就可以解码出原始数据，为了规避少数异常 DataNode 节点，DataAgent 会根据策略及时发送一些 Backup Requests。同样可以看出，整个 Get 流程具备多个高可用的设计。

Aries 的读写流程还有一个特点，即，对**读写链路做了适当的分离**，其好处是可以尽量降低读写的故障干扰，尽可能地缩小故障域。最后，Aries 的读写路径中不包含中心化单点，不会出现单点瓶颈问题。

第二个方面是容错设计。Aries 支持在 DataNode 之间、Rack 之间、DC 之间以及 AZ 之间的精细化副本分布。Aries 可以保证，一个 Rack 内并行升级，或者 Rack 级别出现异常时不会影响读写。同时 Aries 也支持多 AZ 容灾，但是支持多 AZ 容灾它一定是带有条件的。不管是机房建设、还是资源供应都会影响多 AZ 容灾的能力，另一方面，多 AZ 容灾也可能会导致副本数，也就是存储成本上升。总体而言，Aries 有条件地支持多 AZ 容灾。

2.3.2 数据可靠性

数据可靠性是存储系统的生命线。Aries 在这个方向上，相比于此前的一些存储系统，重视程度显著提升，并且针对性地做了大量的工作，包括故障应对、数据修复、数据容灾以及数据校验等。这些工作如下面两张图所示：



第一个方面，故障应对。存储系统面对的故障，更多的是磁盘和机器的故障。对于磁盘故障，首先可以通过百度智能云的 HAS 服务来实现预测或者检测，另外，Aries 程序自身内部也会持续检测 I/O 线程是否 hang 住，以及盘符连续的多块盘同时故障这种存在关联性的异常。**如果磁盘被预测出可能会故障，Aries 会尽快将上面的数据 Balance 出去**，如果盘已经故障，访问总是出错，那么 Aries 会自动踢掉这个坏盘，触发数据修复。当然，自动踢盘也不是盲目的，是有条件地进行并具备兜底保护，确保不会因为随意踢盘导致数据出现严重的丢失风险。除了换盘本身是人工操作外，剩下的上下线流程都是自动化的，效率很高。此外，Aries 的运维平台具备磁盘在线率监控，以关注坏盘处理的过程是否符合预期。

如果出现机器级别的故障，根据不同的机型和集群规模，Aries 处理策略则有所不同。如果是一个超大规模的集群，并且机型的密度不算很高（不超过我们设定的阈值），那么可以直接踢掉机器（同样有相应的兜底保护机制）触发数据修复。如果机器密度很高，或者集群规模太小，则不会立即触发自动踢机器，而是由机器维保人员介入，尽快把机器的坏件进行修复，然后带着数据回归集群，这样做的目的是避免盲目的数据修复。在有备件的情况下，运维流程能保证在 X 天（一般是 2 天）之内完成机器的坏件修复和回归。如果机器超过 Y 时间（不同集群的 Y 值不一样，一般为个位数天数）仍未完成坏件修复和回归，集群也可能直接进入踢机器然后修复数据的流程。当然，即使已经进入数据修复过程，修好的机器一旦带着数据回归，Aries 也能支持将尚未来得及修复的数据直接加载进去，减少修复量。

如果系统出现大规模故障，Aries 会自动进入 Safe Mode 状态，系统处于这个状态时，一部分操作是自动化处理的，另外一部分操作由运维人员及时介入，按照预案进行处理。

第二个方面，数据修复。Aries 的数据修复是自动检测和触发的，在修复机制中，以 Volume 为单位做任务调度，以 Slice 粒度做精确修复。任务调度机制支持优先级，可以保证可靠性风险比较高的数据尽量得到更高优先的修复。

第三个方面，数据容灾。包括 Slice 回收站机制、Master 的元数据备份与恢复以及 DataNode 的元数据重建等。

第四个方面，**数据校验**。Aries 的数据校验机制包括三个层面，分别是实时校验、后台周期性校验和跨系统校验。

实时校验：第一个是**数据在网络中传输时端到端的校验**，第二个是**数据在内存和磁盘之间存取时的校验**，第三个是**DataAgent 在完成数据编码之后立即异步执行的一次解码的校验**，异步解码校验的目的是及时发现 CPU 或者内存这些硬件的异常（虽然概率极小，但出现过）导致的数据出错。

后台周期性校验：包括正确性校验、一致性校验和完整性校验。正确性校验是指校验磁盘上的 Shard 的正确性，确保能够发现静默错误之类的异常。一致性校验是指 Shard 之间的自治性校验，其目的是确认任意 r 个 Shard 都能够解码出原始的 Slice。完整性校验是指校验某个 Slice 是否出现某些 Shard 长期缺失，或者说，确认整个 Slice 是完整的，实施完整性校验的原因在于，不能假定数据修复机制永远不会出现 Bug 或者失效，所以需要另外一个机制对修复机制自身的运行效果进行审计。上述的这些后台周期性校验机制均支持抽查模式，在抽查模式下，系统能够很快地校验完一轮全集群。

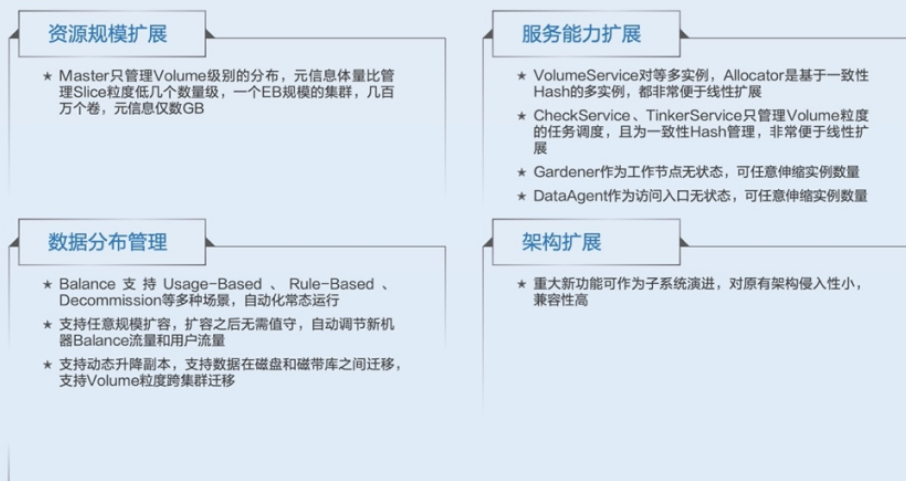
跨系统校验：对于网盘而言，跨系统校验就是在网盘 PCS 层和 Aries 层之间进行校验，主要目的是确保上下两层所看到的数据存在性视图是一致的，确认没有出现数据丢失，同时也能发现一些垃圾数据。

上述图中展示了两张 Aries 线上校验相关的监控图，左边的图是一致性校验的进展监控图，从图中可以看出，各个集群的校验进度在缓慢上行，一般而言，我们会根据业务的要求进行速度调节，确保在一个季度或者半年之内能够完成一轮校验。右边的图是完整性校验的进展监控图，从这个图中可以看出，其校验周期比一致性校验要小很多。

2.3.3 扩展能力

Aries 的扩展能力包含资源规模的扩展、数据分布管理、服务能力的扩展和架构扩展等几个方面，如下图所示：

分布式环境挑战 — 系统的扩展功能



资源规模扩展能力。**Aries 的 Master 只管理 Volume 这个粒度的分布**，由于 Volume 这个容器的粒度很大，所以即使是一个高达 EB 规模的集群，Volume 的元数据也只有几个 GB，可谓是非常之小。或者说，相比于管理 Slice 粒度的分布，集群元信息的总量要低几个数量级，这种方式使得 Aries 不会出现元数据瓶颈，很容易扩展规模。

数据分布管理。第一点是 Balance 机制，Aries 的 Balance 机制支持 Usage-Based（基于容量利用率）、Rule-Based（基于副本分布规则）、Decommission 等多种场景，自动化常态运行。Aries 支持任意规模的扩容，小到一块盘一台机器，大到一大批机器都可以支持。扩容之后无需值守，系统自动调节新磁盘 / 机器上面 Balance 流量和用户流量，尽量在不影响用户读写的情况下加快 Balance。Aries 也支持动态升降副本，支持数据在磁盘和磁带库之间进行迁移，也支持 Volume 的跨集群迁移。

服务能力的扩展。Aries 的几个服务模块中，VolumeService 对等多实例部署，Allocator、CheckService 和 TinkerService 基于一致性 Hash 实现多实例管理和部署，都非常便于线性扩展。DataAgent 和 Gardener 等无状态的工作节点，可任意伸缩实例数量。

架构的扩展性。Aries 的重大新功能，可以作为一个子系统进行演进，在这种方式下，新功能对原有的架构的侵入性比较小，也容易实现良好的兼容性。

2.3.4 资源及管理

Aries 能够应用多种资源，在多个层面针对资源具备良好的管理，如下图所示：

资源管理的挑战



存储介质方面。Aries 支撑网盘以 HDD 磁盘为主，其中 CMR 磁盘以 18T 和 20T 的为主，SMR 磁盘以 20T 的为主，当前也正在引入 22T 的 CMR 盘和 26T 的 SMR 盘。同时，网盘也使用了磁带库，目前磁带使用的是 LTO-8 系列，单盘 12T。

机型方面。已经从早期的 18 盘机型演进到现在主流的 38 盘和 92 盘机型，138 盘机型也已经处于验证中。磁带库单体在 200PB 以上，正在引入磁带柜模式。

资源视图方面。纵向来看，从上往下，分为多个层次。一个集群包含多个 AZ，每个 AZ 下面包含多个 DC，每个 DC 下面包含多个机架，然后每个机架下面包含多个机器，每个机器下面包含多个磁盘。也就是说，Aries 管理物理资源到磁盘的粒度。横向来看，Aries 的资源池对应到一系列的机器，通过 Table Space 绑定资源池的形式，实现逻辑资源和物理资源的映射。

资源调度与运营方面。这个方面包含了大量的相关工作，包括容量预测，资源调配，供应链以及风险跟踪等，还包括硬件维修管理，磁盘在线率管理，机器在线率管理等。

2.3.5 CDP Trade-off: 以网盘为例

以网盘 PCS 为例，介绍一下相关的成本考量及相应的 Trade-off，如下入所示：

网盘PCS的成本考量与Trade-off



成本方面。第一点是空间的节省，Aries 实现了**非常灵活的 EC 模型**，最低可 1.2 副本（不含压缩）。

Aries 也支持压缩，包括通用压缩（Slice 压缩和 Shard 压缩）和图片无损压缩等。Aries 在引擎的设计上也做了一些特殊的优化，比如支持实时回收并利用删除留下的空洞。此外还包含了一些小的优化，比如磁盘格式化参数的优化，由于 Aries 的随机写引擎中 Volumelet 在磁盘上是大型文件，所以磁盘格式化的时候不需要保留过多的 Inode 空间，这样可以减少一些 Inode 空间导致的浪费。

第二点是硬件成本。Aries 机型和介质主要采用大容量低成本的硬件，同时在运营层面保证足够的磁盘在线率和机器在线率，减少浪费。

图中右下角提供了一个表格，这是网盘的一个线上集群的统计。很明显，这个集群非常大，实际有 1.7EB 的物理规模。从表格中可以看出，这个集群的 EC 模式分三种，一种是 2+4 的，一种是 18+6 的，还有一种 27+6 的，分别对应 3.0 副本、1.33 副本和 1.2 副本，其中主要采用的是 1.33 副本，总体而言，这个集群的平均副本数在 1.3 左右。

本小节的最后，介绍一下网盘 PCS 在数据存储上的 Trade-off（这个 Trade-off 只代表数据存储在 Aries 上的 Trade-off，不代表网盘业务自身的考量）。总体而言，**网盘的数据在 Aries 这一层，存储的成本和数据的可靠性的优先级稍高于性能**。那么这些 Trade-off 又是如何体现的？

第一，对于大部分数据，采用比较大的 EC 参数，也就是采用比较大的 r 值，去实现更低的副本数。比如说采用 18+6 的方式，相比于传统的 8+4 的方式，能够实现更低的副本数，**但是因为 r 值变大了，访问 Shard 的时候，会有更大的概率受磁盘长尾的影响，会导致 Slice 的访问延迟上升**。关于副本数，我们认为，不宜过低，比如低至 1.1 甚至 1.05 的副本数并不需要更多的技术，但是过低的副本数，要么会带来性能的抖动或下降，要么会导致数据可靠性的下降。

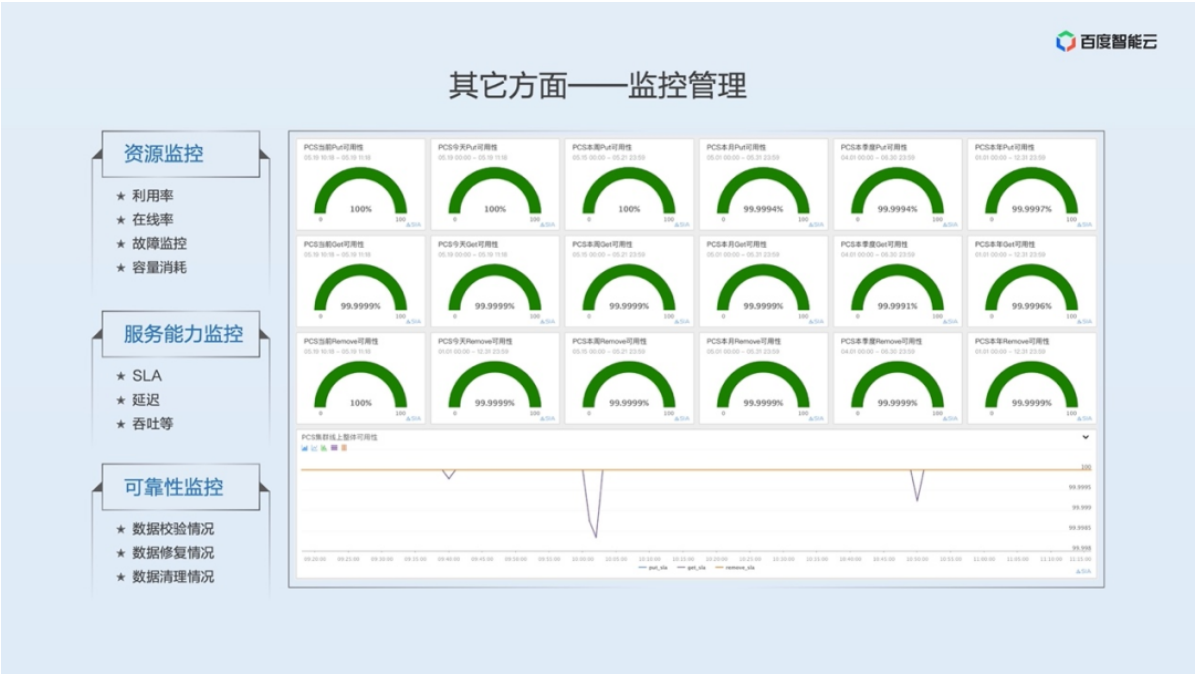
第二，**对于那些特别小的数据，大 EC 参数可能导致访问延迟差异过大**，考虑到这部分数据容量占比非常低，故做了一个反向的折衷，减少 r 值，也就是，数据不要 EC 得太细，同时设置足够的 k 值以保证足够的数据可靠性。也就是说，在保证数据可靠性足够的前提下，通过增加副本数，来降低延迟，提升性能。

第三，引入磁带这种低成本介质来降低成本。但是，磁带是一种线性访问介质，磁带库的硬件构造有其特殊性，这会导致首字节访问延迟至少在分钟级。而且整个磁带库的总吞吐非常低，一旦有大量数据同时访问，会出现排队现象，长尾延迟会受到很大的影响。

第四，在 Aries 的单机引擎中，数据是 DIO 写入的，**相比于写 Page Cache，DIO 的延迟更大，性能会变差**，但在面对极端异常时，数据会更加安全可靠。

2.3.6 监控管理

下图简要展示了 Aries 在监控层面做的工作以及一些效果：

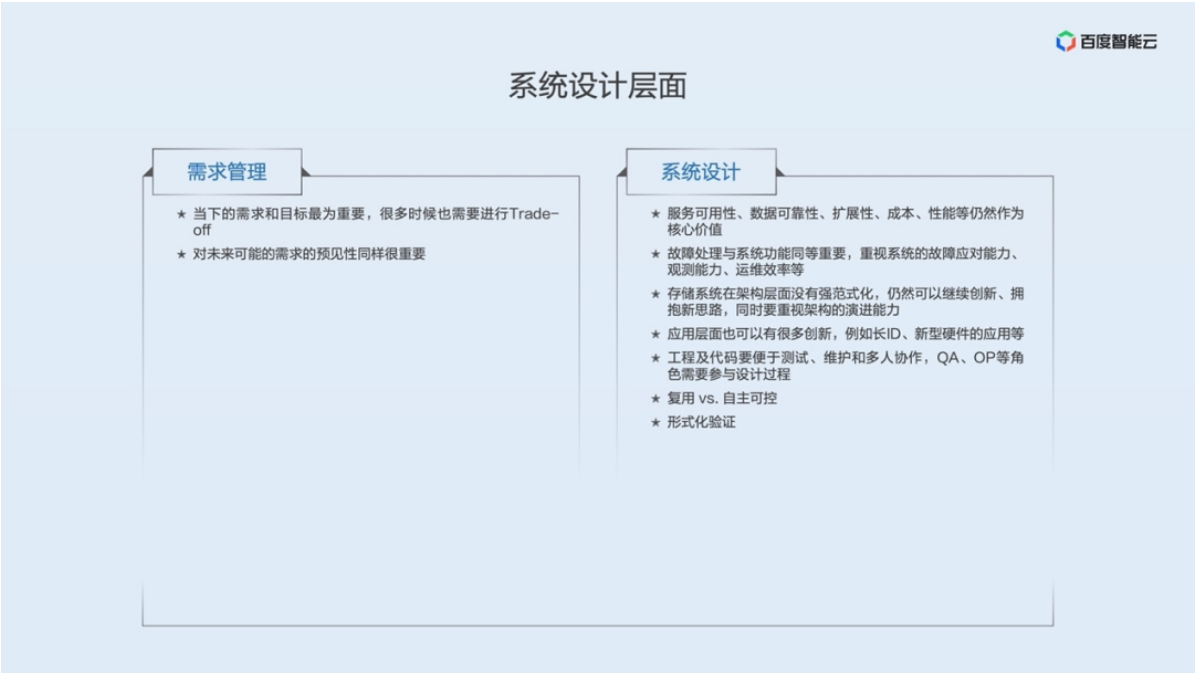


Aries 的监控总体上比较完善的。图中也列举了一些非常核心的监控，比如资源监控、服务能力监控以及一些数据可靠性相关的监控。从实际运行情况来看，监控征具备良好的效果。图中也展示了 Aries 支撑网盘 PCS 的可用性监控大屏，从大屏中可以看出，不管是以天、周、月、季度和年度为周期来统计，也不管是 Put、Get 还是 Remove 接口，基本都实现了五个九的可用性，是非常高的。

百度沧海的经验与思考

这是本文的最后一部分，在这一部分，我们分享一些我们在多年研发存储的过程中的经验与思考，包括系统设计层面和监控与运维两个层面。

系统设计层面经验与思考如下图所示：



需求管理方面。第一，我们认为当下的需求和目标是最为重要的，当然，往往也需要对这些需求进行 Trade-off。第二，对未来可能的需求的预见性同样也很重要，在需求分析的时候，不能仅仅只看当前的需求，还需要看一下未来一年两年甚至三年的需求，进行综合的考量。如果能充分考虑到近两三年可能的需求的话，在设计系统时候容易做到更好的兼容性，也更易于演进。

系统设计方面。第一，我们老生常谈一些东西，像服务可靠性、数据可靠性、扩展性、成本和性能等等，它们仍然是存储系统的核心价值。如果一个存储系统在这些关键点上都不做得不理想，那这个系统的应用价值则会大打折扣，甚至无法被用户和客户认可，研发人员也会缺乏成就感。

第二，这一点也是我们经验之谈，一个系统，**它的故障处理或者故障应对能力应当与系统的功能同等重要**，毕竟这些能力也是保障系统能够长期稳定运行的关键之一，所以务必要重视系统的故障应对能力，提升系统的观测能力，提升系统的运维效率等。

第三，我们认为**存储系统在架构层面目前仍然没有强范式化**，仍然可以继续创新，仍然可以拥抱不同的新思路，在设计系统的时候，也要重视架构的长期演进能力。

第四，我们认为应用层面也可以有很多创新。这里举个例子，比如长 ID 的应用，最开始设计 Aries 的时候，我们参考了文件系统的 Inode 设计，把 Slice ID 设计为 64 位，但是我们逐步发现，64 位的长度捉襟见肘，使得一些问题非常难以解决，所以最后将 Slice ID 的长度从 64 位扩展为 128 位，ID 中新增了多种信息，不仅使得我们很快解决了遇到的设计难点问题，而且还带来了一些其它的好处。又比如一些新型硬件的应用，有些问题，客观来看，用硬件解决，比用软件解决要高效很多，在这种情况下，适当增加一点成本来通过硬件解决问题可能是更加事半功倍的。

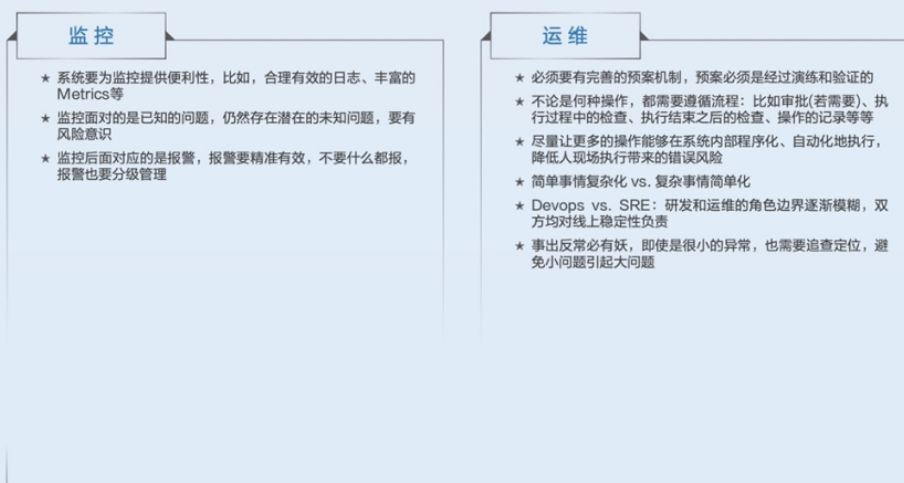
第五，工程和代码一定要便于测试，便于其他人来维护，便于多人协作开发。除了研发角色之外，测试和运维这样的角色，也需要在设计过程中参与进来，而不是待工程完成了研发之后测试才介入，待系统上线了运维才介入。这些角色提前参与到设计阶段，有利于让系统充分反映各方的建议和期望，使设计更加完善。

第六，技术复用和自主可控的考量，不管是在设计还是研发阶段，不管是从整个系统层面还是其中或大或小的组件的层面，架构师都需要充分考虑是复用业界 / 同行 / 同事做得比较优秀技术，抑或是要自行研发保证自主可控。

第七，形式化验证，形式化验证是业界近几年开始比较重视的方向，当今的系统，逻辑越来越复杂，设计的时候靠工程师的经验去推理异常场景，这固然很重要，但难免会有遗漏，如果能通过形式化验证的方法去对复杂的逻辑进行一个完整的验证，有助于提前发现问题，提前规避异常风险，非常有利于系统未来能更稳定地运行。

监控和运维层面的经验与思考如下图所示：

监控运维层面



监控方面。第一，务必认识到监控是非常重要的，系统一定要为监控提供便利性。首先，系统一定要提供有合理有效的日志，通过日志能够获知系统各种各样的运行状态。进一步，只有日志肯定也是不够的，也需要能够提供相应的机制对系统的运行状态进行各种观测，这需要有完善的指标采集、汇聚、统计和展示的能力。

第二，监控面对的是已知的问题，作为一个复杂的分布式系统，仍然可能存在一些现有监控尚未覆盖到的潜在未知问题，一定要有风险意识，不能因为系统上线跑了三五年没再出过新问题就掉以轻心，对线上环境要有敬畏之心。

第三，监控后面对应的是报警，报警一定要精准有效，不要什么都报，否则一天收个几百条报警，有效报警和无效报警各种混杂在一起，使得人很容易遗漏重要信息，而且时间长了之后，容易对报警产生不再重视的意识，从而埋下隐患。所以报警一定要有分级管理机制，什么时候只报 IM 消息即可，什么时候该报短信，什么时候该报邮件，什么时候该直接打电话都需要仔细确定，报警接收人也需要清楚何种级别的报警对应何种处理方式。

运维方面。第一，运维层面必须要提供完善的预案机制，而且预案都必须是经过演练和验证的。比如 Aries 系统，有非常多的预案，都经过了演练，形成了非常明确和完善的预案手册。

第二，不论是何种操作，都需要遵循相应的流程。比如有些操作可能比较关键，需要审批，那就先走审批流程。在实施操作时，在操作执行过程当中需要做检查，执行结束之后也需要做检查以确认是否符合预期等，操作做完之后需要做好记录方便以后回溯，更不用说由该操作导致了故障的场景。

第三，尽量让更多的操作能够在系统内部程序化、自动化地执行，降低由人在现场做线上操作出错触发异常的风险，这一点也要求系统在设计的时候，充分考虑可运维性。

第四，简单事情复杂化和复杂事情简单化的的考量。简单事情复杂化是指，有些操作本身特别简单，但可能带来很严重的后果，所以要把它的执行变得更加复杂，避免“不小心”出错。比如一个强制删除数据的操作，一条命令即可完成，操作很简单，但是一旦出错后果很严重，可能需要执行过程中加一个二次确认等。复杂事情简单化是指，比如一些操作过程包含很多步骤，步骤之间是纯流水线式执行的，而且不会带来严重后果，对于这类操作，把它的所有步骤封装起来一键执行，或者让系统内部能够自动化一次性执行，不仅能提高效率，也能减少出错的概率。

第五，Devops 和 SRE (Site Reliability Engineer)，这是这几年业界出现和讨论得比较多的两个词汇。Devops 是指从开发走向测试、交付和运维的循环迭代方法，SRE 是站点稳定性工程师，SRE 隐含的意义是指，作为稳定性工程师，不仅只是做做线上部署与操作、安装监控、处理线上问题这么简单，还需要能够参与到稳定性相关的各个环节，甚至在设计与研发阶段，也能做出自己的贡献。从这两个词汇中，我们可以看到一个趋势，即研发和运维等角色，其边界是在逐步模糊的，这些角色之间不再是严格孤立的，需要有机地合作，各方都对线上稳定性负责。

第六，也是最后一点，“事出反常必有妖”，线上环境不管出现什么样的异常，即使它很小，都不应该被忽略，而需要我们去定位清楚，找到根因。有果必有因，如果不能及时定位出根因，后续有可能会造成更大的问题。实际经验表明，很多大问题在爆发之前，往往会有很多先兆，及时发现并解决这些小的先兆问题，有利于降低出现大问题的风险。